Studio Nx

Backend(FastAPI/Python) 56 — ProjectCreateRequest · ProjectUpdateRequest

2026-10-12

Backend(FastAPI/Python) 56 — `ProjectCreateRequest` · `ProjectUpdateRequest` (프로젝트 요청 스키마)

항목	내용
파일	`studionx/backend/app/schemas/project.py`
작업	P1-13 프로젝트 API: 만들기와 수정 요청 본문 검사 (PRD 01 FR-122, FR-124)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	55 `retention.main()` · 57 `get membership` · `require role`

1. 하는 일

새 프로젝트 모달과 Edit details 모달이 보낸 값을 **저장하기 전에** 검사합니다. 값이 틀리면 422 `VALIDATION_ERROR`로 응답합니다 (P1-03 오류 처리기).

보낸 값	결과
`{"title": " My Song "}`	통과합니다. 제목은 `"My Song"`, 설명은 없음, BPM은 120, 박자는 4/4가 됩니다
제목이 `""`, `" "`, 101자	422
설명이 `" "`	통과합니다. 설명은 **없음**(None)으로 저장됩니다
박자 `7/8` (`time_sig_num=7, time_sig_den=8`)	통과합니다
박자 아래 숫자가 `3`, `12`, `64`	422입니다. 1, 2, 4, 8, 16, 32만 허용합니다
수정 요청 `{"title": null}`	422입니다. 제목은 비울 수 없습니다
수정 요청 `{}`	통과합니다. 아무것도 바꾸지 않습니다
수정 요청 `{"description": ""}` 또는 `null`	통과합니다. 설명을 지웁니다

2. 언제 쓰나

요청	스키마	화면
`POST /api/v1/projects`	`ProjectCreateRequest`	myProject.html의 새 프로젝트 모달 (P1-14)
`PATCH /api/v1/projects/{id}`	`ProjectUpdateRequest`	Edit details 모달 (P1-14)

FastAPI는 라우터 인자의 타입을 보고 자동으로 검사합니다. 그래서 서비스(58, 59)는 이미 검사와 정리가 끝난 값을 받습니다.

3. 코드

import와 상수 (파일 맨 위)

```
from datetime import datetime
from decimal import Decimal
```

```
from enum import StrEnum
from typing import Annotated

from pydantic import BaseModel, Field, StringConstraints, field_validator

PROJECT_PAGE_SIZE_MAX = 50  # 한 번에 받을 수 있는 최대 개수 (화면은 10)
PROJECT_TITLE_MAX = 100  # 화면 #new-title · #edit-title maxlength 와 같음
PROJECT_DESCRIPTION_MAX = 500  # 화면 #new-description · #edit-description maxlength 와 같음
TIME_SIG_DENOMINATORS = (1, 2, 4, 8, 16, 32)  # 박자 아래 숫자 (음표 길이)

ProjectTitle = Annotated[
    str, StringConstraints(strip_whitespace=True, min_length=1, max_length=PROJECT_TITLE_MAX)
]
ProjectDescription = Annotated[
    str, StringConstraints(strip_whitespace=True, max_length=PROJECT_DESCRIPTION_MAX)
]
```

`Annotated`, `StringConstraints`, `field_validator`와 상수, 타입 4개를 이번에 추가했습니다.

만들기

```
class ProjectCreateRequest(BaseModel):
    title: ProjectTitle
    description: ProjectDescription | None = None
    bpm: Decimal = Field(default=Decimal("120"), ge=20, le=999, decimal_places=2)
    time_sig_num: int = Field(default=4, ge=1, le=32)
    time_sig_den: int = Field(default=4)

    @field_validator("description")
    @classmethod
    def _empty_to_none(cls, value: str | None) -> str | None:
        return value or None  # 공백만 친 설명 → "" → None

    @field_validator("time_sig_den")
    @classmethod
    def _check_den(cls, value: int) -> int:
        if value not in TIME_SIG_DENOMINATORS:
            raise ValueError("must be one of 1, 2, 4, 8, 16, 32")
        return value
```

`bpm`과 `time_sig_num` 줄은 템플릿을 그대로 두었습니다.

수정

```
class ProjectUpdateRequest(BaseModel):
    title: ProjectTitle | None = None
    description: ProjectDescription | None = None

    @field_validator("title")
    @classmethod
    def _title_not_null(cls, value: str | None) -> str:
        # 보내지 않으면 이 검사는 돌지 않음 (기본값은 검사하지 않음) → null 을 직접 보낸 경우만
        if value is None:
            raise ValueError("title cannot be empty")
        return value
```

```
    @field_validator("description")
    @classmethod
    def _empty_to_none(cls, value: str | None) -> str | None:
        return value or None
```

문법 풀이

import 문법	뜻
`from typing import Annotated`	타입에 검사 규칙을 붙여서 새 이름(`ProjectTitle`)으로 만듭니다
`from pydantic import StringConstraints, field_validator`	앞은 글자 규칙(공백 제거, 길이)이고, 뒤는 칸 하나를 직접 검사하는 함수입니다

식	뜻
`StringConstraints(strip_whitespace=True, min_length=1, …)`	앞뒤 공백을 **먼저** 지운 다음 길이를 잽니다
`ProjectTitle | None = None`	제목 또는 null이고, 보내지 않으면 None입니다
`@field_validator("time_sig_den")` + `@classmethod`	그 칸의 기본 검사가 끝난 뒤에 불리는 함수입니다 (pydantic 규칙상 classmethod로 씁니다)
`raise ValueError(...)`	검사에 실패했다는 뜻이며, pydantic이 이것을 422로 바꿉니다
`value or None`	`""`처럼 빈 값이면 None, 아니면 그 값을 그대로 씁니다
`model_dump(exclude_unset=True)`	(59에서 사용) **보낸 칸만** 꺼냅니다. 보내지 않은 칸은 바꾸지 않기 위해서입니다

4. 흐름

1. JSON을 받아 칸마다 타입과 범위를 검사합니다 (BPM 20~999, 소수 둘째 자리까지, 박자 위 숫자 1~32).
2. 제목과 설명은 앞뒤 공백을 지운 다음 길이를 검사합니다.
3. `field_validator`가 빈 설명을 None으로 바꾸고, 박자 아래 숫자를 확인하고, 수정 요청에서 제목 null을 거부합니다.
4. 하나라도 틀리면 422로 응답하고, 모두 통과하면 서비스로 넘깁니다.

5. 왜 이렇게

선택	이유
제목을 **공백을 지운 뒤** 검사	템플릿(`min_length=1`)대로 두면 공백 한 칸짜리 제목이 저장됩니다. 닉네임(37), 문의(50)와 같은 빈 틈입니다. 그래서 서비스에 있던 `data.title.strip()`(템플릿)은 필요 없어졌습니다
빈 설명은 None으로	칸을 비우는 것은 "설명 없음"을 뜻합니다. `""`와 `NULL` 두 가지가 섞이지 않게 했습니다. Edit details에서 설명을 지우면 화면은 빈 문자열을 보냅니다
박자 아래 숫자는 6개만 허용	음표 길이(온음표, 2분, 4분, 8분, 16분, 32분음표)만 박자의 아래 숫자가 될 수 있습니다 (템플릿 TODO). 화면은 4/4, 3/4, 6/8, 7/8만 고를 수 있지만, 서버에서도 막습니다
수정 요청의 제목 null 거부를 **스키마**에서 처리	템플릿은 서비스에서 422를 내라고 했지만, 스키마에서 막으면 다른 검사 오류와 같은 모양(`VALIDATION_ERROR` + `details`)으로 응답합니다
길이 상수를 화면과 같게 맞춤	화면의 `maxlength`(100, 500)와 서버 기준이 어긋나지 않게 했습니다 (문의 50과 같은 방식)

6. 시험 (`tests/test_project_schemas.py` 새 파일, 33개)

시험	확인한 것
기본값	제목만 보내면 설명은 None, BPM은 120, 박자는 4/4입니다
공백	제목과 설명의 앞뒤 공백을 지우고, 공백을 지운 뒤 100자인 제목은 통과합니다
틀린 제목 4가지	`""`, 공백만, 줄바꿈만, 101자는 422입니다
설명	빈 칸과 공백은 None이 되고, 501자는 422입니다
박자	아래 숫자 1, 2, 4, 8, 16, 32는 통과하고 0, 3, 5, 12, 64, −4는 422입니다. 위 숫자 0과 33도 422입니다
BPM	19.99, 999.01, 소수 셋째 자리는 422입니다
수정	빈 요청은 바꿀 것이 없고, 제목 null, 빈 제목, 101자 제목은 422입니다. 공백이 섞인 제목은 정리되고 빈 설명은 None이 됩니다

전체 시험 438개가 통과했고, `ruff` 검사도 통과했습니다.